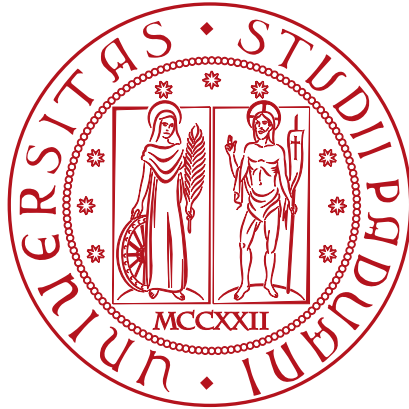


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

CORSO DI LAUREA IN INFORMATICA



**Migrazione al protocollo MQTT per un
sistema di monitoraggio basato su
dispositivi RFID**

Tesi di Laurea Triennale

Relatore

Prof. Marco Zanella

Laureando

Luca Ribon

Matricola 2075516

Cit...

— Autore

Dedicato a ...

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, della durata di circa 320 ore, dal laureando Luca Ribon presso l'azienda KanbanBOX S.r.l.

L'obiettivo finale dello stage è stato quello di sostituire il protocollo HTTP, utilizzato dalla piattaforma KanbanBOX per comunicare con i lettori RFID, con MQTT, un protocollo più adatto ad essere utilizzato con dispositivi IoT.

In particolare è stata progettata un'architettura capace di integrare i seguenti componenti:

- **lettore RFID**, dispositivi hardware che leggono i tag RFID applicati sui prodotti
- **AWS IoT Core**, un servizio cloud che integra un broker capace di gestire la comunicazione tra molteplici dispositivi tramite MQTT
- **AWS SQS**, un servizio cloud che fornisce delle code di messaggi per la comunicazione asincrona tra servizi, particolarmente adatto a gestire la lettura dei tag tramite meccanismo publish/subscribe di MQTT
- **KanbanBOX**, la piattaforma web che necessita di ricevere i dati letti dai lettori RFID per monitorare e supportare l'intero processo produttivo e logistico di un'azienda

A tal fine, alla fase di progettazione è seguita la codifica delle funzionalità necessarie in KanbanBOX per l'integrazione con i servizi AWS IoT Core e SQS, insieme all'implementazione della comunicazione con i lettori tramite protocollo MQTT.

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine al Prof. Marco Zanella relatore della mia tesi, per l'aiuto e il sostegno fornitomi durante tutto il processo di stesura.

Desidero ringraziare con affetto la mia famiglia per il sostegno, il grande aiuto e per essermi stati vicini in ogni momento durante gli anni di studio.

Infine tengo a ringraziare tutti i miei amici per aver contribuito a rendere memorabile ogni momento passato assieme, che fosse di svago o produttivo.

Padova, Mese 2026

Luca Ribon

Indice

1. Introduzione	1.
1.1. Organizzazione del testo	1.
1.2. L'azienda	1.
1.3. Il progetto	1.
2. Descrizione dello stage	4.
2.1. Contesto aziendale	4.
2.2. Metodo di lavoro	4.
2.2.1. Metodologia Agile e Scrum???	4.
2.2.2. Git e GitHub	4.
2.2.3. Sentry	5.
2.2.4. PhpStorm	5.
2.2.5. Microsoft Teams	5.
2.3. Analisi dei rischi	6.
2.3.1. Rischi organizzativi	6.
2.3.1.1. Gestione del tempo non ottimale	6.
2.3.2. Rischi tecnici	6.
2.3.2.1. Inesperienza con tecnologie usate in azienda	6.
2.3.2.2. Influenze esterne	6.
2.3.3. Rischi di analisi e progettazione	6.
2.3.3.1. Variazione dei requisiti	6.
2.3.3.2. Scelte progettuali non adeguate	6.
3. Analisi dei requisiti	8.
3.1. Attori	8.
3.2. Casi d'uso	8.
3.3. Tracciamento dei requisiti	9.
3.4. Panoramica funzionalità	9.
4. Introduzione teorica	10.
4.1. Tecnologie utilizzate	10.
4.2. Aspetti teorici rilevanti	10.
4.3. Strumenti scelti	10.
5. Descrizione del lavoro svolto	12.
5.1. Ciclo di vita del software	12.
5.2. Progettazione	12.
5.3. Design Pattern utilizzati	12.
5.4. Codifica	12.

6. Conclusioni	14.
6.1. Consuntivo finale	14.
6.2. Raggiungimento degli obiettivi	14.
6.3. Conoscenze acquisite	14.
6.4. Probabili margini di miglioramento	14.
6.5. Considerazioni personali	14.
 Bibliografia	 16.
 Glossario	 18
P	18
Parola	18

Elenco delle Figure

Figura 1 Use Case - UC0: Scenario principale	8.
--	----

Elenco delle Tabelle

Tabella 1 Tabella del tracciamento dei requisiti funzionali	9.
Tabella 2 Tabella del tracciamento dei requisiti funzionali	9.
Tabella 3 Tabella del tracciamento dei requisiti funzionali	9.

Capitolo 1.

Introduzione

1.1. Organizzazione del testo

Il documento è strutturato in sei capitoli principali:

1. Introduzione: describe.
2. Descrizione dello stage: describe.
3. Analisi dei requisiti: describe.
4. Introduzione teorica: describe.
5. Descrizione del lavoro svolto: describe.
6. Conclusioni: describe.

Riguardo la stesura del testo, sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune vengono definiti nel glossario, situato alla fine del presente documento; per la prima occorrenza dei termini riportati nel glossario viene utilizzato il seguente stile:
- i termini in lingua straniera o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*.

1.2. L'azienda

KanbanBOX S.r.l. è un'azienda con sede a Vicenza che si occupa dello sviluppo, della commercializzazione e della consulenza relativi alla piattaforma web KanbanBOX.

KanbanBOX è uno strumento progettato per supportare le aziende nell'implementazione di metodologie Lean nella gestione della produzione e della logistica delle aziende.

Più nello specifico, la piattaforma consente di tracciare e gestire lo stato dei materiali grezzi, semilavorati e finiti, durante le fasi di approvvigionamento e produzione. Ciò è reso possibile attraverso l'utilizzo di cartellini Kanban elettronici, che permettono di identificare e tracciare i prodotti man mano che vengono consumati o resi disponibili dai processi aziendali.

1.3. Il progetto

KanbanBOX prevede due metodi di lettura dei cartellini Kanban:

- scansione di **codici a barre**: per ogni cartellino vengono stampata un'etichetta su cui sono presenti dei codici a barre che possono essere scansionati tramite appositi lettori ottici o tramite la fotocamera di comuni dispositivi mobili;
- lettura di chip **RFID**: per ogni cartellino viene stampata un'etichetta RFID, ovvero un'etichetta dotata di un chip che può essere letto da apposite antenne RFID.

In entrambi i casi le operazioni svolte sui cartellini vengono comunicate a KanbanBOX tramite delle API RESTful che sfruttano il protocollo HTTP per la trasmissione dei dati. In particolare, nel caso della lettura tramite RFID, il lettore RFID mette a disposizione delle API HTTP, eseguite in locale sul lettore stesso, che consentono la comunicazione con esso.

I modelli di lettori RFID utilizzati da KanbanBOX supportano anche la comunicazione tramite protocollo MQTT, un protocollo di comunicazione del livello applicativo, basato su un meccanismo *publish/subscribe*; in aggiunta i lettori implementano nativamente un metodo di connessione sicuro e pratico al servizio AWS IoT Core.

MQTT è particolarmente adatto per la comunicazione con dispositivi *IoT*, in quanto progettato per essere meno oneroso in termini di consumo di banda e risorse computazionali rispetto a HTTP; inoltre prevede dei meccanismi di *Quality of Service (QoS)* e di memorizzazione dei messaggi che lo rendono più affidabile in scenari in cui la connettività di rete non è stabile.

Per questi motivi il progetto di stage si è focalizzato sulla sostituzione del protocollo HTTP con MQTT per la trasmissione dei tag letti tra lettori RFID e KanbanBOX con l'obiettivo di migliorare l'affidabilità e l'efficienza della comunicazione; inoltre è stata prevista l'implementazione di funzionalità di configurazione dei lettori direttamente all'interno di KanbanBOX, operazioni che in precedenza dovevano essere eseguite manualmente tramite l'interfaccia web fornita dal produttore del lettore.

Ho scelto questo progetto perché prometteva di approfondire e di maturare esperienza con temi e tecnologie che mi incuriosiscono, come l'Internet of Things, i protocolli di comunicazione e le infrastrutture cloud. Un altro fattore interessante è stato il contatto diretto e concreto con il prodotto su cui ho lavorato che ho potuto vedere all'opera in un contesti reali.

Capitolo 2.

Descrizione dello stage

2.1. Contesto aziendale

Lo stage si è svolto in presenza presso la sede di Vicenza di KanbanBOX S.r.l. Sono stato inserito nel team sviluppatori software con il quale ho, fin da subito, partecipato a tutte le attività di routine previste, le più rilevanti sono:

- **daily stand-up meeting:** riunione giornaliera di circa 15 minuti in cui ogni membro del team condivide lo stato di avanzamento del proprio lavoro, le difficoltà incontrate e gli obiettivi per la giornata;
- **Dev+ weekly:** riunione settimanale del team di sviluppo a cui partecipa anche un consulente esterno, con esperienza rilevante nel mondo PHP open source, con cui si affrontano i problemi più ostici e si analizzano approfonditamente potenziali soluzioni;
- **Analisi issue Sentry:** riunione settimanale tra tutti i membri del team in cui si analizza il report delle issue tracciate da Sentry, uno strumento di monitoraggio degli errori in produzione; l'obiettivo dell'incontro è quello di identificare i bug più critici e frequenti e di assegnarli ai membri del team per la risoluzione.

Per tutta la durata dello stage ho avuto a disposizione il supporto del mio tutor aziendale, il quale mi ha guidato nell'inserimento all'interno del team e del prodotto KanbanBOX su cui ho lavorato.

Inoltre, ho avuto modo di collaborare anche con altri membri del team per discutere con chi aveva maggiore esperienza in merito agli argomenti o alle tecnologie con cui ero meno familiare.

2.2. Metodo di lavoro

2.2.1. Metodologia Agile e Scrum???

2.2.2. Git e GitHub

Per il **versionamento** di tutta la codebase e la documentazione di KanbanBOX viene utilizzato **Git**, un sistema di controllo di versione distribuito.

I repository Git sono ospitati su **GitHub**, una piattaforma di hosting per progetti software che fornisce funzionalità aggiuntive usate dal team di KanbanBox come la gestione delle issue, le pull request con relative verifiche del codice e l'integrazione continua.

Nel repository di KanbanBOX, ovvero quello contenente tutta la codebase del prodotto, viene usato il flusso di lavoro **Git Flow**, che prevede l'utilizzo di due rami principali, **master** e **development**.

In **master** viene mantenuta sempre la versione stabile e in produzione, mentre in **development** viene costruito lo *snapshot* della prossima *release*. In aggiunta a questi due rami principali, ne esistono altri con scopi specifici che seguono la seguente nomenclatura:

- **feature/nome-feature**: rami usati per lo sviluppo di nuove funzionalità;
- **fix/nome-fix**: rami usati per la risoluzione di bug;
- **hotfix/nome-hotfix**: rami usati per la correzione di bug critici in produzione;
- **chore/nome-chore**: rami usati per attività di manutenzione o che in generale non apportano modifiche percebili dall'utente finale (aggiornamenti di librerie, configurazioni, implementazione di funzionalità esistenti che non prevedono risoluzione di bug).

Quando lo scopo di un ramo viene assolto interamente, viene aperta una **pull request** su GitHub per richiedere la revisione e l'integrazione delle modifiche nel ramo **development** (o **master** nel caso di hotfix).

Quando si individuano problemi o nuove esigenze internamente al team vengono create delle **issue** su GitHub dai membri interessati seguendo un modello predefinito che ne facilita la categorizzazione e la prioritizzazione.

Oltre che dagli sviluppatori, anche i membri del team dei consulenti applicativi utilizzano i repository su GitHub per svolgere le attività di supporto ai clienti; infatti, ogni qualvolta viene aperta una richiesta di assistenza o di una nuova funzionalità da parte di un cliente, viene creata un'issue su GitHub in cui vengono documentati i dettagli della richiesta ricevuta.

2.2.3. Sentry

Sentry è una piattaforma di **monitoraggio degli errori** che consente di tracciare, analizzare e risolvere i problemi nelle applicazioni in tempo reale.

All'interno di KanbanBOX viene utilizzato sia per ricevere notifiche automatiche sugli errori che si verificano in produzione, sia per gestire in modo più ordinato e collaborativo gli errori che si verificano aggiungendo dettagli utili ai bug su cui si sta lavorando e categorizzandoli.

Giornalmente viene scelto un membro del team di sviluppo che si dedicherà maggiormente a monitorare e analizzare le issue segnalate da Sentry in quella giornata. In più, settimanalmente viene organizzata una riunione dove il responsabile Sentry di quella giornata, assieme al resto del team, analizza e discute le issue più frequenti dell'ultimo periodo con l'obiettivo di categorizzarle, documentarle nel modo più dettagliato possibile e assegnarle ai membri del team per la risoluzione.

2.2.4. PhpStorm

Per lo sviluppo di KanbanBOX viene utilizzato PhpStorm, un **ambiente di sviluppo integrato** (IDE) specifico per il linguaggio PHP che offre funzionalità avanzate come l'analisi statica del codice, refactoring automatizzato, il debugging e l'integrazione con sistemi di controllo di versione come Git.

2.2.5. Microsoft Teams

Per la **comunicazione** interna al team e con il resto dell'azienda viene utilizzato Microsoft Teams, una piattaforma di collaborazione che integra chat, videoconferenze, condivisione di file e integrazione con altre applicazioni Microsoft 365.

All'interno dell'azienda sono presenti sia canali generali per la comunicazione tra tutti i dipendenti, sia canali specifici per ogni team di lavoro. Inoltre sono presenti canali dedicati ad attività specifiche come le riunioni Dev+ weekly e le analisi delle issue di Sentry (vedi Sezione 2.1).

2.3. Analisi dei rischi

I rischi individuati e analizzati nella fase iniziale del progetto si possono suddividere in tre categorie principali:

- rischi organizzativi;
- rischi tecnici;
- rischi di analisi e progettazione.

2.3.1. Rischi organizzativi

2.3.1.1. Gestione del tempo non ottimale

Descrizione: durante lo svolgimento dello stage potrebbe non essere sempre possibile gestire il tempo in modo ottimale a causa di attività aziendali ausiliarie o imprevisti.

Soluzione: pianificazione intelligente degli appuntamenti secondari in modo che non vadano a interferire con le attività principali dello stage.

2.3.2. Rischi tecnici

2.3.2.1. Inesperienza con tecnologie usate in azienda

Descrizione: durante lo svolgimento dello stage potrebbero presentarsi dei rallentamenti dovuti alla mia inesperienza con alcune tecnologie o metodologie usate in azienda.

Soluzione: dedicare la fase iniziale alla comprensione approfondita delle tecnologie ed interfacciarsi con i membri del team per chiedere supporto in caso di difficoltà bloccanti.

2.3.2.2. Influenze esterne

Descrizione: durante lo svolgimento dello stage potrebbero sorgere problemi tecnici dovuti a fattori esterni all'azienda, come ad esempio bug o disservizi di dipendenze esterne del prodotto.

Soluzione: analizzare il problema per identificare il prima possibile la reale causa, successivamente cercare di mitigarlo, se necessario coinvolgendo altri membri del team con maggiore esperienza; nel caso in cui il problema risultasse non risolvibile internamente ripianificare le attività in modo da minimizzare l'impatto sullo svolgimento dello stage.

2.3.3. Rischi di analisi e progettazione

I rischi appartenenti a questa categoria derivano principalmente dal fatto che le tecnologie usate non sono ancora approfonditamente esplorate dal team di sviluppo, quindi è necessaria una fase di studio e sperimentazione per comprenderne al meglio le potenzialità, i limiti e le opzioni di integrazione con il prodotto.

2.3.3.1. Variazione dei requisiti

Descrizione: durante la fase di studio potrebbero emergere nuove informazioni che potrebbero essere incongruenti con le aspettative o rendere obsoleti i requisiti inizialmente definiti.

Soluzione: mantenere una comunicazione costante con il tutor aziendale per discutere eventuali cambiamenti nei requisiti.

2.3.3.2. Scelte progettuali non adeguate

Descrizione: durante la fase di integrazione e sviluppo potrebbero emergere nuove esigenze, limitazioni od opportunità date dalle tecnologie scelte.

Soluzione: intervallare le fasi di integrazione o sviluppo con momenti di studio e sperimentazione mirati in modo da accertarsi che le scelte fatte siano adeguate, sfruttando anche le nuove informazioni derivanti dalle fasi di integrazione o sviluppo di altre tecnologie, svolte precedentemente.

Capitolo 3.

Analisi dei requisiti

Breve introduzione al capitolo

3.1. Attori

3.2. Casi d'uso

Per lo studio dei casi di utilizzo del prodotto sono stati creati dei diagrammi. I diagrammi dei casi d'uso (in inglese *Use Case Diagram*) sono diagrammi di tipo *UML* (**TODO AGGIUNGI GLOSSARIO** in qualche modo) dedicati alla descrizione delle funzioni o servizi offerti da un sistema, così come sono percepiti e utilizzati dagli attori che interagiscono col sistema stesso. Essendo il progetto finalizzato alla creazione di un tool per l'automazione di un processo, le interazioni da parte dell'utilizzatore devono essere ovviamente ridotte allo stretto necessario. Per questi motivi i diagrammi dei casi d'uso risultano semplici e in numero ridotto.

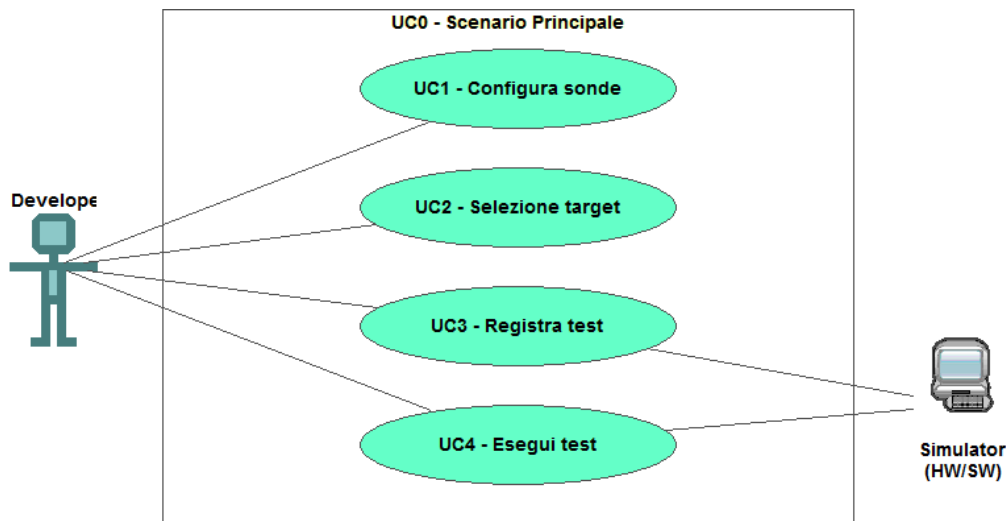


Figura 1: Use Case - UC0: Scenario principale

UC0: Apertura plugin

Attore principale	Sviluppatore applicativi
Precondizioni	Lo sviluppatore è entrato nel plug-in di simulazione all'interno dell'IDE
Postcondizioni	Il sistema è pronto per permettere una nuova interazione

Scenario principale La finestra di simulazione mette a disposizione i comandi per configurare, registrare o eseguire un test

3.3. Tracciamento dei requisiti

Da un'attenta analisi dei requisiti e degli use case effettuata sul progetto è stata stilata la tabella che traccia i requisiti in rapporto agli use case.

Sono stati individuati diversi tipi di requisiti e si è quindi fatto utilizzo di un codice identificativo per distinguerli.

Il codice dei requisiti è così strutturato R(F/Q/V)(N/D/O) dove:

R = requisito
F = funzionale
Q = qualitativo
V = di vincolo
N = obbligatorio (necessario)
D = desiderabile
Z = opzionale

Nelle tabelle Tabella 1, Tabella 2 e Tabella 3 sono riassunti i requisiti e il loro tracciamento con gli use case delineati in fase di analisi.

Requisito	Descrizione	Use Case
RFN-1	L'interfaccia permette di configurare il tipo di sonde del test	UC1

Tabella 1: Tabella del tracciamento dei requisiti funzionali

Requisito	Descrizione	Use Case
RQD-1	Le prestazioni del simulatore hardware deve garantire la giusta esecuzione dei test e non la generazione di falsi negativi	–

Tabella 2: Tabella del tracciamento dei requisiti funzionali

Requisito	Descrizione	Use Case
RVQ-1	La libreria per l'esecuzione dei test automatici deve essere riutilizzabili	–

Tabella 3: Tabella del tracciamento dei requisiti funzionali

3.4. Panoramica funzionalità

Capitolo 4.

Introduzione teorica

4.1. Tecnologie utilizzate

4.2. Aspetti teorici rilevanti

4.3. Strumenti scelti

Breve introduzione al capitolo

Capitolo 5.

Descrizione del lavoro svolto

Breve introduzione al capitolo

5.1. Ciclo di vita del software

5.2. Progettazione

5.3. Design Pattern utilizzati

5.4. Codifica

Capitolo 6.

Conclusioni

6.1. Consuntivo finale

6.2. Raggiungimento degli obiettivi

6.3. Conoscenze acquisite

6.4. Probabili margini di miglioramento

6.5. Considerazioni personali

Bibliografia

Glossario